POSTECH · FIAP — TECH CHALLENGE FASE 02 · ENTREGA INDIVIDUAL

ToggleMaster: containerização e orquestração em Kubernetes

5 microsserviços no Amazon EKS, com RDS, ElastiCache, DynamoDB, SQS e autoscaling por HPA

IDENTIFICAÇÃO

Nome	Diego Barbosa Brandão Rocha
RM	375936
Discord	coddp

LINKS DA ENTREGA

Código	**LINK DO REPOSITÓRIO GITHUB**
Vídeo	**LINK DO VÍDEO (até 20 min)**
Originais	github.com/FIAP-TCs

1 Resumo da solução

Os cinco microsserviços do ToggleMaster (auth, flag, targeting, evaluation e analytics) foram conteinerizados, validados localmente com Docker Compose e publicados em um cluster Amazon EKS, com autoscaling por HPA validado sob carga.

Cliente → **Nginx Ingress (NLB)** → **evaluation-service** → **Redis · flag · targeting**

evaluation-service → **Fila SQS** → **analytics-service** → **DynamoDB**

2 Conteinerização

- **Dockerfiles multi-stage** nos 5 serviços. Go (auth, evaluation): build em `golang:1.22-alpine` e imagem final `distroless/static nonroot`. Python (flag, targeting, analytics): dependências em estágio separado, imagem final `python:3.11-slim`, usuário não-root (uid 10001) e gunicorn.
- **docker-compose.yml:** 5 aplicações + 2 PostgreSQL + Redis + DynamoDB Local (9 contêineres). Um override adiciona o ElasticMQ como emulador de SQS, para rodar sem conta AWS.
- **Validação local de ponta a ponta:** criação de chave de API, flag e regra, avaliação e gravação do evento no DynamoDB Local.

3 Infraestrutura na AWS

AWS Academy Learner Lab · região us-east-1 · LabRole

RECURSO	CONFIGURAÇÃO
EKS	Cluster `togglemaster` (Kubernetes 1.37), node group gerenciado `tm-nodes`: 2 × t3.medium (mín. 1, desejado 2, máx. 4)
ECR	5 repositórios (um por serviço), imagens com a tag `v1`
RDS PostgreSQL	3 instâncias: `tm-auth`, `tm-flags`, `tm-targeting` (db.t3.micro)
ElastiCache	1 cluster Redis `tm-redis` (cache.t3.micro)
DynamoDB	Tabela `ToggleMasterAnalytics`, chave de partição `event_id` (String)
SQS	Fila Standard `togglemaster-queue`

Rede	Security Group dos dados libera 5432 e 6379 apenas a partir do Security Group do cluster
Entrada	Metrics Server (add-on) e Nginx Ingress (Helm) com Network Load Balancer

4 Kubernetes

- **5 Namespaces**, um por serviço (auth, flags, targeting, evaluation, analytics).
- **Deployments** com imagem do ECR, requests/limits de CPU e memória e probes de readiness e liveness (`/health`).
- **Services** ClusterIP, **ConfigMaps** e **Secrets** em base64. Os valores reais são gerados por script a partir de variáveis de ambiente e não ficam no repositório.
- **Ingress (Nginx)** com as rotas `/auth` , `/flags` , `/targeting` e `/evaluate` .
- **Jobs** de inicialização dos três bancos RDS (`CREATE TABLE IF NOT EXISTS`).
- **Scripts:** `build-push.sh` , `gen-secrets.sh` , `deploy.sh` e `aws-academy-infra.sh` .

5 Escalabilidade (HPA)

SERVIÇO	MÉTRICA	ALVO	RÉPLICAS
evaluation-service	CPU	70%	mín. 2 · máx. 6
analytics-service	CPU	50%	mín. 1 · máx. 5

> **Opção A (AWS Academy).** O KEDA exigiria um role IAM (IRSA) para ler a profundidade da fila SQS, o que o Learner Lab não permite (somente a LabRole). Foi usado HPA por CPU: com a fila cheia, o analytics processa mais mensagens, a CPU sobe e o HPA escala.

Teste de carga (`hey` , 3 min, 100 conexões simultâneas, endpoint `/evaluate`):

119.431	663	144 ms	74 mil+
requisições, todas com status 200	requisições por segundo	latência mediana (p99: 258 ms)	eventos gravados no DynamoDB

- **evaluation-service:** de 2 para 6 réplicas, com a CPU em cerca de 494% do alvo.
- **analytics-service:** escalou ao máximo de 5 réplicas (CPU em ~594% do alvo) para drenar a fila, que chegou a mais de 50 mil mensagens pendentes.

6 Escolha dos bancos

SERVIÇO DE DADOS	USADO POR	MOTIVO
RDS PostgreSQL	auth, flag, targeting	Dados relacionais e transacionais com esquema definido; um banco por microsserviço.
ElastiCache Redis	evaluation	Cache de leitura de baixa latência no caminho crítico, com TTL, evitando consultar flag e targeting a cada requisição.
DynamoDB	analytics	Alto volume de escrita de eventos, sem esquema fixo e sem joins.

7 Dificuldades encontradas e soluções

1. **Plano gratuito da conta pessoal:** não lançava instâncias t3.medium (tipo não elegível ao Free Tier) e os nós nunca subiam. Migração para o AWS Academy.

2. **EKS Auto Mode:** o cluster nasceu com o Auto Mode, que criou um acesso do tipo `EC2` para a LabRole, e os nós gerenciados não conseguiam se registrar. Solução: desligar o Auto Mode, instalar os add-ons (vpc-cni, kube-proxy, coredns, metrics-server) e recriar o acesso como `EC2_LINUX`.
3. **Credenciais sem IRSA:** os pods não encontravam credenciais AWS porque o IMDS dos nós tinha hop limit 1. Solução: aumentar para 2, e os pods passaram a usar a LabRole do nó.
4. **Código-fonte original:** erros de build (imports não usados em Go, go.mod inválido, dependências incompatíveis e endpoint do SQS fixo), corrigidos para compilar e permitir um emulador de SQS local.
5. **Compatibilidade de versões:** Werkzeug fixado em 2.2.3 e boto3 em 1.34.162.

8 Como reproduzir

Detalhes em `docs/GUIA-FASE2.md`. Resumo:

```
# 1) fila SQS e tabela DynamoDB
./scripts/aws-academy-infra.sh base
# 2) criar o cluster EKS e o node group (Console)
# 3) RDS ×3 e ElastiCache
./scripts/aws-academy-infra.sh data
# 4) imagens no ECR, Nginx Ingress via Helm e deploy
./scripts/build-push.sh
kubectl apply -f k8s/00-namespaces.yaml
./scripts/gen-secrets.sh
./scripts/deploy.sh
```

Após a gravação do vídeo, os recursos da AWS devem ser removidos para evitar consumo de créditos (`aws-academy-infra.sh destroy`, exclusão do node group, do cluster EKS e dos repositórios ECR).